

Architecture options for multi-plant PI reel data

Objective: manage multiple plants through a common data infrastructure, standardise canonical variable names and units, store about 400 summarised variables per reel in Amazon S3, and retain access to raw time-series data for online inference.

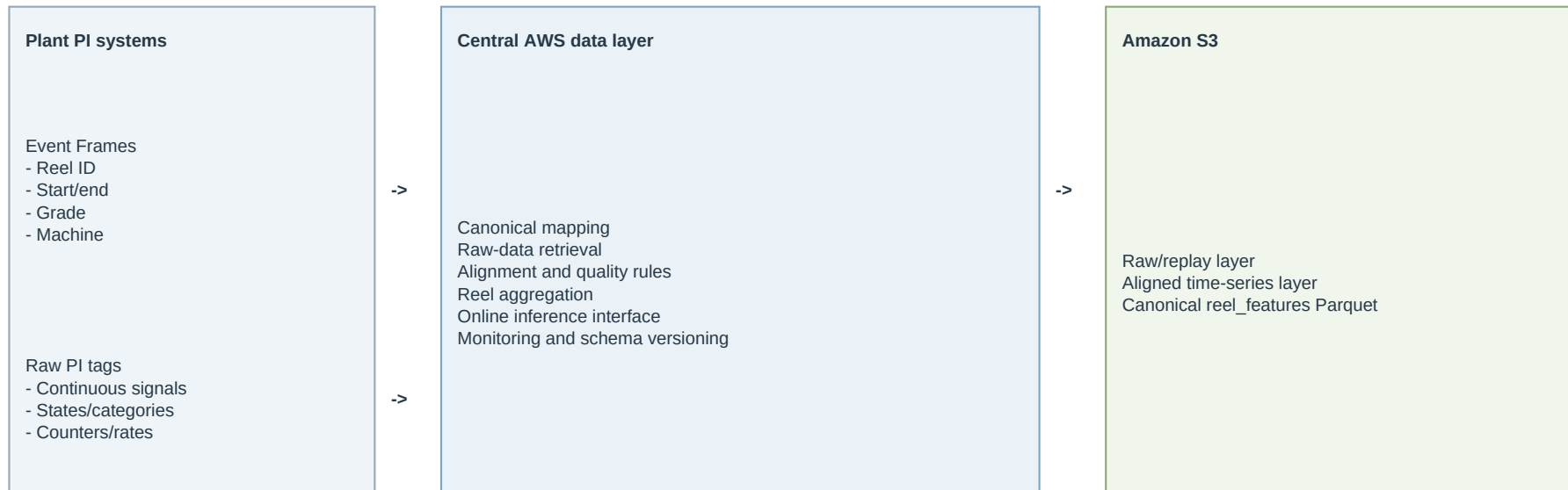
Recommended default: use Event Frames as the authoritative reel segmentation and a central AWS pipeline to retrieve raw PI time series, calculate the common feature set, and publish canonical Parquet datasets to S3.

Ranked comparison

Rank	Architecture	Description	Requirements / Configuration	Pros	Cons
1	Event Frames + central AWS time-series pipeline	Use Event Frames only for authoritative reel ID and start/end boundaries. Retrieve raw PI time series for the combined interval, calculate the ~400 summaries centrally, and write Parquet to S3. The same raw-data access can support online inference.	Requirements: PI Web API connectivity; reliable Event Frames; plant-specific tag mapping; common aggregation engine; checkpointing; retries; S3 and monitoring. Configuration: Canonical variable registry with local PI path, unit conversion, data type, interpolation rule, quality filter and reel aggregation. Schedule extraction and define late-data policy.	Best cross-plant standardisation; one codebase; authoritative boundaries; full control of Parquet layout; easy recalculation; supports online inference and replay.	You own scheduling, state, retries, alignment and monitoring. More raw data is transferred and PI load must be managed.
2	PI Integrator for summaries + PI Web API for online data	Configure an Event View per plant to calculate and publish one row per closed reel to S3. Use PI Web API separately for raw time-series access required by online inference.	Requirements: PI Integrator licensing and deployment; Event Frames; S3 target; PI Web API for inference; consistent governance across plants. Configuration: Event View with ~400 columns and their summary rules; plant-specific source mappings; continuous publication interval; S3 credentials and target settings.	No custom CronJob for summary publication; scheduling, incremental publishing and recovery are handled within PI; low custom-code burden for stable datasets.	Two data paths; duplicated PI configuration across plants; dependence on PI administrators; less control over file format/partitioning; historical recalculation may be harder.
3	AWS job requesting PI summaries per Event Frame	Query newly closed Event Frames and ask PI Web API to calculate the configured summary for each tag and reel, using StreamSet and Batch where possible. Write the resulting rows to S3.	Requirements: Event Frame search; PI Web API; tag mapping; scheduled AWS task; checkpointing and deduplication; separate raw-data access for inference. Configuration: Per-variable summary type, unit and filter; batch size, pagination, retry policy, timeout, overlap window, S3 schema and correction policy.	Less raw-data transfer; central summary definitions; no need for 400 Event Frame attributes; direct Parquet output.	Work grows with reels x variables; potentially heavy PI summary workload; many logical API operations; summary and online-inference paths remain separate.

Rank	Architecture	Description	Requirements / Configuration	Pros	Cons
4	Reel-ID tag + central raw time-series pipeline	Retrieve a reel/product identifier tag with the process signals, infer reel boundaries from identifier changes, calculate summaries locally, and publish to S3.	Requirements: Reliable reel-ID tag; stateful segmentation; raw time-series access; plant mapping; validation against official production records. Configuration: Rules for transitions, open reels, downtime, reused IDs, resampling, interpolation, counters, late data and deduplication.	No dependency on Event Frames or PI Integrator; one central pipeline can serve aggregation and online inference; highly flexible.	Reimplements official reel logic; boundary mismatches can distort totals; high data volume; complex timestamp alignment and categorical handling.
5	Store ~400 captured summaries inside Event Frames	Configure PI AF analyses to calculate and capture roughly 400 attributes when each reel closes. Export the flattened Event Frames to S3 using PI Integrator or an AWS job.	Requirements: Large Event Frame template; hundreds of AF attributes and analyses; PI sizing; capture/backfill strategy; S3 export mechanism; separate raw-data access for inference. Configuration: Source, formula, units, quality rules, capture timing and recalculation policy for every attribute, plus export configuration.	Very simple downstream extraction; summaries are visible in PI and can be treated as governed operational records.	Highest PI-side maintenance; AF database growth; difficult changes and backfills; duplication in PI and S3; poor fit for evolving ML features; does not solve online inference.

Recommended target architecture



Why this ranks first

- Event Frames preserve the official reel boundaries instead of recreating them from a tag.
- A central mapping layer standardises names, units, valid ranges and aggregation definitions without forcing plants to rename local PI tags.
- The same raw-data access supports online inference, historical replay and recalculation of reel features.
- The full analytical schema can evolve without adding hundreds of attributes to PI AF or rebuilding Event Views at every plant.
- S3 layout, Parquet schema, versioning and quality controls remain under central governance.

Canonical variable registry

Field	Purpose	Example
canonical_name	Shared variable name across plants	steam_flow
plant_source	Local PI Point or AF attribute	\\PLANT_A\PM1_STEAM
data_type	Continuous, state, category, counter or rate	rate
unit_in / unit_out	Normalise units centrally	kg/h -> t/h
reel_aggregation	Calculation applied over the official reel interval	integral
online_required	Whether the signal is needed by online inference	true
quality_rule	Good-value, range and machine-state filtering	good and MachineRunning

Decision guidance

Choose	When it is the best fit
Event Frames + central AWS time-series pipeline	You prioritise cross-plant standardisation, one centrally governed feature definition, raw data for online inference, flexible Parquet output and reproducible recalculation.
PI Integrator + PI Web API	All plants already operate PI Integrator consistently, the 400-variable schema is stable, local PI teams can maintain mappings, and avoiding a custom summary scheduler outweighs having two data paths.
AWS summary API	Raw-data transfer must be minimised and PI can efficiently calculate the summaries, while the number of reels and backfill requirements remain moderate.
Reel-ID segmentation	Official Event Frames are missing or unreliable, but a stable product identifier tag exists and has been validated against production records.
400 Event Frame attributes	The values are stable, operationally important KPIs that must be visible and governed inside PI - not merely evolving machine-learning features.

Suggested implementation sequence

1	Establish the common contract	Define the canonical variable registry, required Event Frame fields, units, data types, quality rules and schema version.
2	Run a limited proof of concept	Use one plant, one week of Event Frames and 10-20 representative variables, including continuous, categorical, rate and counter signals.
3	Validate against official figures	Compare reel boundaries, production totals and selected operational KPIs with existing PI or reporting outputs.
4	Scale the feature registry	Expand to approximately 400 variables only after PI load, runtime, missing-data behaviour and S3 schema have been verified.
5	Onboard additional plants	Add plant-specific source mappings while keeping canonical names, calculations and output contracts unchanged.

Recommended hybrid policy: keep only a limited set of stable official KPIs in PI Event Frames, while generating the full ~400-variable analytical feature set centrally and storing it in S3.